

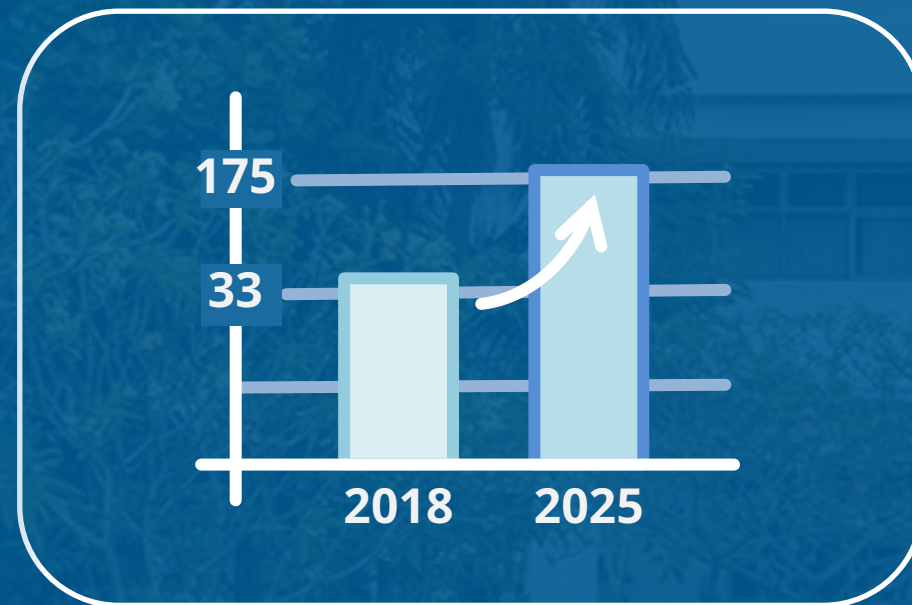


Analisis Kinerja Arsitektur Integrasi Data ETL, ELT, dan Hibrida pada *Data Warehouse* Berbasis *PostgreSQL*

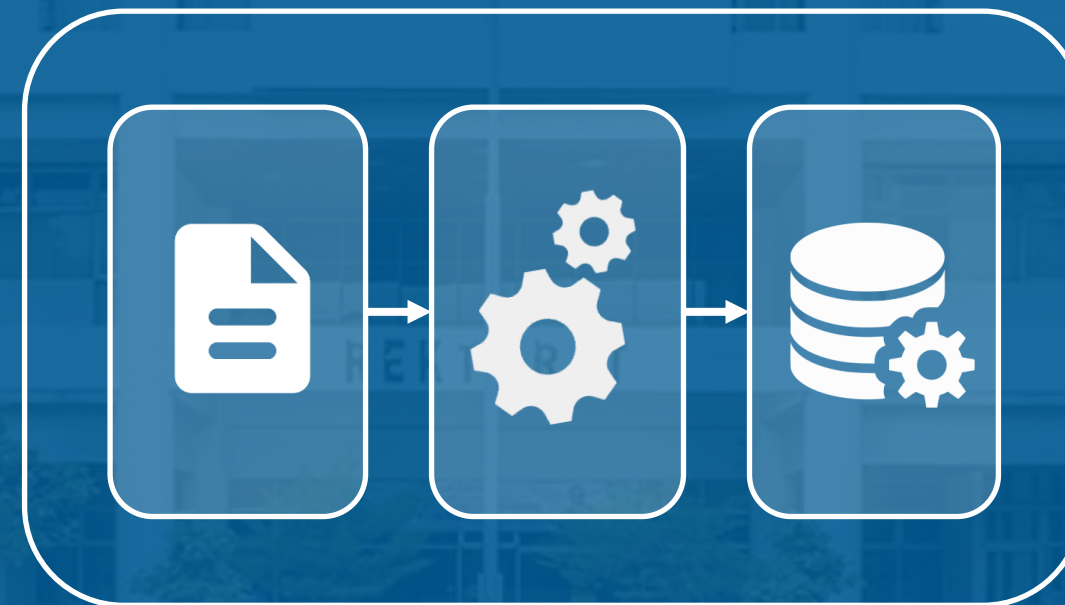


Oleh:
Luthfan Aryananda Purwito

Apa yang melatarbelakangi tugas akhir ini?



Lonjakan masif pada **volume** dan **varietas** data (terstruktur, semi-terstruktur, dan tidak terstruktur)



Organisasi dituntut memiliki strategi integrasi data yang **efisien** dan **cepat** untuk pengambilan keputusan



ETL menghadapi **isu skalabilitas** dan **kinerja** saat menangani data bervolume besar



ELT menawarkan **performa lebih tinggi** dengan memanfaatkan kekuatan komputasi warehouse.



Hibrida menggabungkan **kontrol ETL** dan **kecepatan ELT**.



Studi komparatif komprehensif antara ketiga arsitektur ini masih terbatas...



...terutama implementasi skenario *on-premise* dengan variasi format data (CSV, JSON, Parquet)



Penelitian Terdahulu

(PERBANDINGAN)

Penelitian	ETL	ELT	Hibrida	Kuantitatif	Format data
Farhan dkk. (2024)	✓	✓	✓	✓	✗
Rambabu dkk. (2023)	✓	✓	✗	✗ (kualitatif)	✗
Haryono dkk. (2020)	✓	✗	✗	✗ (survei)	✗
Rongala (2025)	✓	✓	✗	✓	✗
Sivabalan & Minu (2021)	✓	✗	✗	✓	✗
Al Khozi (2025)	✓	✓	✗	✓	✗
TA ini	✓	✓	✓	✓	✓

Rumusan Masalah

- 01 Bagaimana perbandingan **waktu pemrosesan total** antara arsitektur ETL, ELT, dan hibrida dalam menangani beban kerja dengan bentuk data terstruktur dan semi-terstruktur?
- 02 Bagaimana perbandingan **efisiensi penggunaan sumber daya** pada ketiga arsitektur tersebut selama proses integrasi data berlangsung?
- 03 Bagaimana perbandingan **efisiensi penggunaan ruang penyimpanan disk** pada ketiga arsitektur tersebut pada tahap penyimpanan data mentah dan setelah dimuat ke data warehouse?
- 04 Bagaimana perbandingan **skalabilitas (scalability)** dari ketiga arsitektur tersebut dalam merespons peningkatan volume secara signifikan?



Tujuan

- 01 Untuk membandingkan **waktu pemrosesan total** antara arsitektur ETL, ELT, dan hibrida dalam menangani beban kerja dengan bentuk data terstruktur dan semi-terstruktur
- 02 Untuk membandingkan **efisiensi penggunaan sumber daya** pada ketiga arsitektur tersebut selama proses integrasi data berlangsung
- 03 Untuk membandingkan **efisiensi penggunaan ruang penyimpanan disk** pada ketiga arsitektur tersebut pada tahap penyimpanan data mentah dan setelah dimuat ke data warehouse
- 04 Bagaimana perbandingan **skalabilitas (scalability)** dari ketiga arsitektur tersebut dalam merespons peningkatan volume secara signifikan?

Batasan Masalah



- 01 **Dataset TPC-H** yang dikonversi menjadi dua jenis:
 - Terstruktur (CSV, Parquet)
 - Semi-Terstruktur (JSON)
- 02 **Metrik Pengujian:**
 - Waktu pemrosesan total
 - Penggunaan sumber daya komputasi (*shuffle*, *spill*, dan memori eksekusi Spark)
 - Penggunaan ruang penyimpanan
- 03 **Platform dan teknologi:**
 - Data Warehouse: PostgreSQL
 - Arsitektur ETL: Apache Spark (PySpark)
 - Arsitektur ELT: Apache Spark + dbt
 - Arsitektur Hibrida:
 - Extract, Clean, Load dengan Apache Spark
 - Transform dengan dbt

Landasan Teori



- 01 **ETL**
 - Transform sebelum load: data diolah di mesin pemroses, lalu dimuat ke warehouse
- 02 **ELT**
 - Load sebelum transform: data mentah dimuat dahulu
 - Transformasi dijalankan di dalam warehouse
- 03 **Hibrida**
 - Pendekatan selektif: transformasi ringan (mis. pruning) di awal
 - Transformasi berat diselesaikan di warehouse
 - Menyeimbangkan beban komputasi & I/O

Rancangan Eksperimen



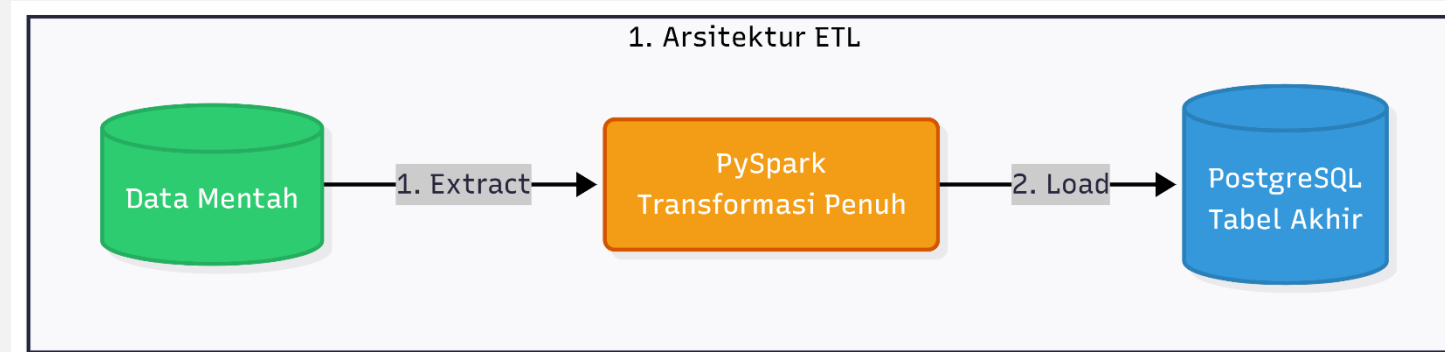
- 01 **Lingkungan**
 - MacBook Air M3 • 8-core • RAM 8 GB • macOS
- 02 **Perangkat & Tools**
 - PostgreSQL 18.4 sebagai data warehouse
 - Apache Spark (PySpark) 4.0.3 • dbt 1.10.22
 - TPC-H DBGEN • Python + DuckDB (konversi format)
- 03 **Dataset & Beban Kerja**
 - TPC-H skala SF5 (~5 GB) & SF10 (~10 GB)
 - Format: CSV, JSONL, Parquet
 - Kueri uji: TPC-H Query 9 — 175 baris, 6 dari 8 tabel
 - Matriks $3 \times 3 \times 2 \times 3$ replikasi = 54 pengujian

Desain Arsitektur



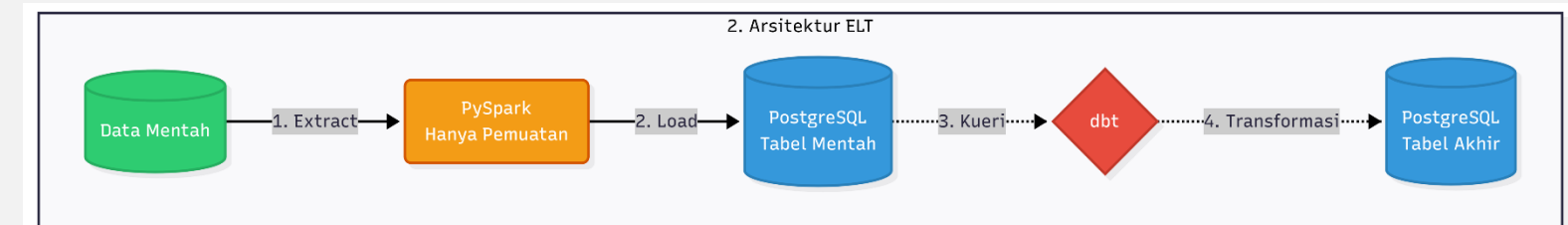
01 ETL

- Transformasi penuh di Apache Spark; hanya 175 baris hasil dimuat ke PostgreSQL



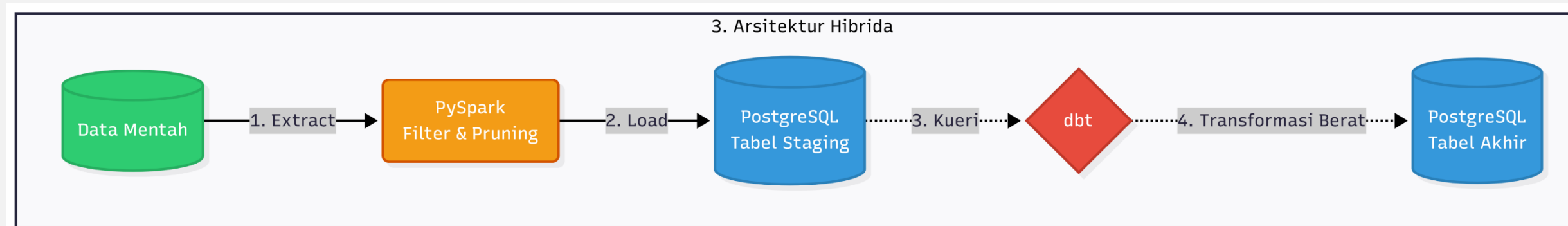
02 ELT

- Spark memuat seluruh kolom mentah ke PostgreSQL via JDBC
- Transformasi (Query 9) oleh dbt di dalam basis data



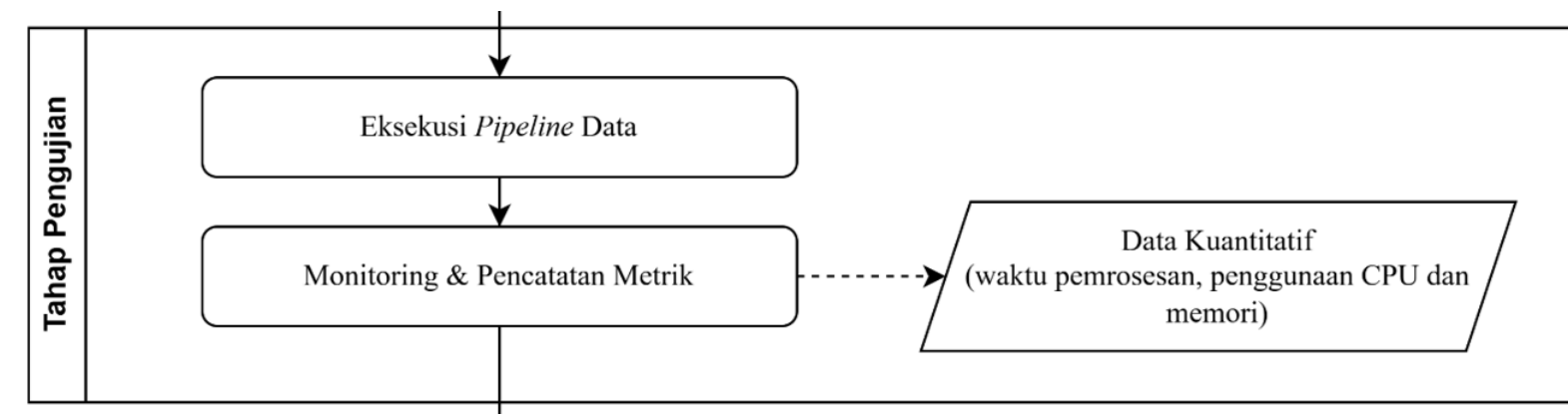
03 Hibrida

- Spark extract-clean-load: column pruning + type casting
- Transformasi akhir oleh dbt di PostgreSQL
- Menyeimbangkan kecepatan komputasi & efisiensi penyimpanan





Skenario Pengujian



Eksekusi *Pipeline*

Faktor	Level yang Diuji
Arsitektur	ETL · ELT · Hibrida
Format Data	CSV · JSONL · Parquet
Skala Data	SF5 (5 GB) & SF10 (10 GB)

Monitoring & Pencatatan Hasil

- **Waktu Pemrosesan** (detik)
- Pengujian **Efisiensi Sumber Daya**

Waktu Pemrosesan Total

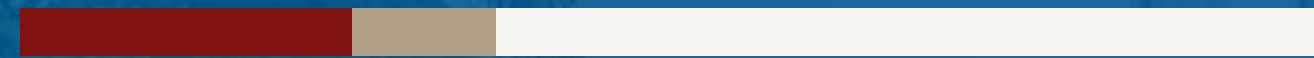


Hasil waktu pemrosesan total (SF5)

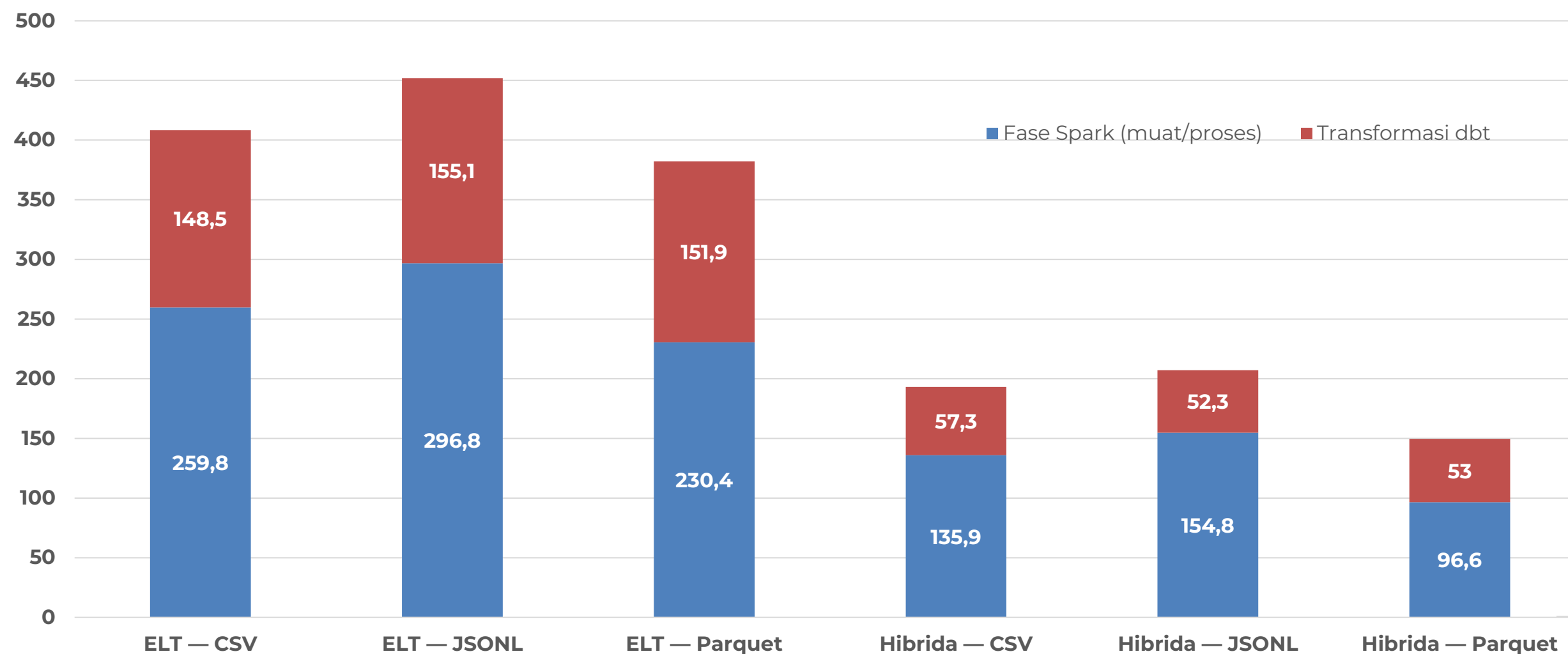
Arsitektur	CSV (s)	JSONL (s)	Parquet (s)
ETL	32,49	57,01	12,81
ELT	408,26	451,88	382,33
Hibrida	193,19	207,10	149,60

- Urutan konsisten di semua format:
ETL < **Hibrida** < **ELT**
- Paling cepat: ETL–Parquet (**12,81** s).
- Paling lambat: ELT–JSONL (**451,88** s)
- Hibrida ± 2 kali lebih cepat dari ELT pada setiap format

Rincian Waktu per Fase



Rincian Waktu Pemrosesan per Fase (SF5)



Meskipun ELT & Hibrida menjalankan kueri SQL Query 9 yang identik di dalam PostgreSQL, durasi transformasi dbt pada **ELT** tercatat sekitar **2,7 hingga 3 kali lebih lama** dibanding **Hibrida**.

Penggunaan Sumber Daya Komputasi

Arsitektur	Fase	Shuffle	Spill (Memori / Disk)	Peak Execution Memory
ETL	Semua fase	225–1.370 MB	Terjadi pada skenario CSV	88-112 MB
ELT	Extract-Load	0	0	0
	Transform	–	–	–
Hibrida	Extract-Clean-Load	0	0	0
	Transform	–	–	–

- **Arsitektur ETL** menunjukkan karakteristik sebagai proses yang paling intensif terhadap sumber daya komputasi (*compute-bound*)
- Hanya arsitektur ETL yang menghasilkan **aktivitas shuffle**, disertai penggunaan **peak execution memory** yang merupakan **tertinggi** di antara seluruh skenario

Penggunaan Ruang Penyimpanan

Sebelum dimuat ke warehouse (SF5):

Format	Volume (MB)
CSV	5.327
JSONL	28.095
Parquet (ETL & Hibrida)	590
Parquet (ELT)	1.563

Setelah dimuat ke warehouse (SF5):

Arsitektur	Tabel mentah di DW (MB)
ETL	~0 (hanya 175 baris)
ELT	7.292
Hibrida	2.451

Temuan Utama:

Format sangat berpengaruh

JSONL = 28 GB, CSV = 5,3 GB

(~5,3 kali lipat)

Arsitektur menentukan jejak warehouse

ETL < Hibrida < ELT

Column-pruning terlihat di 2 titik

ETL/Hibrida
ELT

2,6x

Hibrida
ELT

34%

Penggunaan Ruang Penyimpanan

Tabel Sumber	<i>raw.elt_*</i> (ELT)	<i>raw.hybrid_*</i> (Hibrida)
lineitem	5.280 MB	1.954 MB
orders	1.146 MB	259 MB
partsupp	704 MB	169 MB
part	154 MB	67 MB
supplier	8,4 MB	1,8 MB
Nation	16 kB	16 kB
Total	~7.292 MB (7,3 GB)	~2.451 MB (2,5 GB)

- Perbedaan ini paling menonjol pada tabel lineitem, di mana **ELT** menyimpannya sebesar 5.280 MB, sedangkan **hibrida** hanya 1.954 MB, yaitu sekitar **63% lebih kecil**.
- Hibrida hanya menyimpan 6 dari 16 kolom lineitem yang benar-benar dibutuhkan Query 9

Analisis Pengaruh Format Data

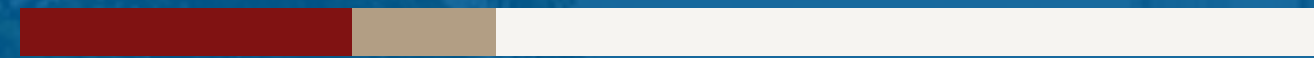
CSV vs Parquet

Arsitektur	CSV (detik)	Parquet (detik)	Parquet lebih cepat (%)
ETL	32,49	12,81	60,6%
ELT	408,26	382,33	6,3%
Hibrida	193,19	149,60	22,6%

CSV vs JSONL

Arsitektur	CSV (detik)	JSONL (detik)	Overhead JSONL (%)
ETL	32,49	57,01	75,5%
ELT	408,26	451,88	10,7%
Hibrida	193,19	207,10	7,2%

Analisis Skalabilitas



Arsitektur	Format	SF5 (detik)	SF10 (detik)	Rasio SF10/SF5
ETL	CSV	32,49	59,08	1,82
	JSON	57,01	111,17	1,95
	Parquet	12,81	23,05	1,80
ELT	CSV	408,26	825,46	2,02
	JSON	451,88	952,22	2,11
	Parquet	382,33	926,87	2,42
Hibrida	CSV	193,19	414,40	2,15
	JSON	207,10	469,87	2,27
	Parquet	149,60	334,89	2,24

Rasio di atas dua menandakan pertumbuhan dengan kinerja memburuk lebih cepat daripada pertambahan data, dan rasio di bawah dua menandakan pertumbuhan sub-linear

Kesimpulan

Metrik	Kesimpulan
Waktu	- ETL konsisten menjadi yang tercepat, diikuti Hibrida, lalu ELT. - Hibrida $\pm$ 2 kali lebih cepat dari ELT berkat <i>column pruning</i>.
Bottleneck	Pada ELT dan Hibrida, hambatan utama ada di pemuatan data mentah via JDBC, bukan pada transformasi.
Sumber Daya	- ETL <i>compute-bound</i> (<i>shuffle</i> & memori Spark tinggi) - ELT & Hibrida I/O- dan <i>database-bound</i>.
Format	- Parquet paling efisien (waktu dan ruang) — juga karena kolom dipangkas. - JSONL paling boros.
Skalabilitas	ETL menskala sub-linear, keunggulannya justru melebar saat volume data bertambah.

Pemilihan arsitektur harus mempertimbangkan kapasitas komputasi mesin target dibanding mesin proses, serta volume data yang dipindahkan antar-komponen. Pada lingkungan *on-premise single-node*, ETL memberikan kinerja yang terbaik.



Saran

Saran	Keterangan
Skala lebih besar	Pengujian pada SF 50++ dengan penyimpanan memadai untuk memverifikasi tren skalabilitas & batas memori ETL (JSONL SF10 sempat terkendala ruang disk).
Lingkungan terdistribusi	Uji pada Data Warehouse terdistribusi, karena keunggulan ELT secara teori bergantung pada komputasi mesin basis data target.
Variasi beban kerja	Gunakan <i>query</i> TPC-H lain di luar <i>Query</i> 9 agar hasil tidak terbatas pada satu pola beban kerja.
Variasi konfigurasi	Uji pengaruh konfigurasi perangkat keras dan Spark yang berbeda terhadap kinerja tiap arsitektur.



Curious about the implementation details?
Feel free to reach me out!

 luthfan@gmail.com

 luthfan.aryananda

 luthfan.ap